

Transformer Architectures and Variants

A Mathematical and Architectural Treatment

Part 3

Efficient, Hierarchical, Memory-Augmented, and Long-Context Transformers

Contents

1	The Long-Context Problem	1
2	Transformer-XL	3
2.1	Why Transformer-XL Was Introduced	3
2.2	Input Segmentation	3
2.3	Memory Construction	4
2.4	Concatenated Memory and Current State	4
2.5	Relative Positional Encoding	5
2.6	What Changed Relative to the Baseline	5
2.7	Applications	5
3	Compressive Transformer	6
3.1	Motivation	6
3.2	Two Memory Levels	6
3.3	Compression Function	6
3.4	Attention over Multiple Memories	7
3.5	Information Flow	7
3.6	Applications	8
4	Reformer	9
4.1	Motivation	9
4.2	Locality-Sensitive Hashing Attention	9
4.3	Attention after Hashing	9
4.4	Reversible Residual Layers	10
5	Routing Transformer	11
5.1	Motivation	11
5.2	Token Representations	11
5.3	Cluster-Based Attention	11
5.4	Difference from Local Attention	12
6	BigBird	13
6.1	Motivation	13
6.2	Local Attention	13
6.3	Random Attention	13
6.4	Global Attention	14

6.5	Why the Combination Matters	14
6.6	Applications	14
7	Perceiver	15
7.1	Motivation	15
7.2	Input and Latent Array	15
7.3	Cross-Attention	15
7.4	Latent Self-Attention	16
7.5	Iterative Processing	17
7.6	Applications	17
8	Perceiver IO	18
8.1	Motivation	18
8.2	Latent Processing	18
8.3	Output Queries	18
9	Funnel Transformer	20
9.1	Motivation	20
9.2	Pooling	20
9.3	Attention after Downsampling	20
9.4	Decoder Reconstruction	21
9.5	Applications	21
10	Synthesizer	22
10.1	Motivation	22
10.2	Random Synthesizer	22
10.3	Dense Synthesizer	22
11	Efficient Transformer Design as a Spectrum	24
11.1	Memory-Based Architectures	24
11.2	Sparse Attention	24
11.3	Latent Bottleneck Architectures	24
11.4	Hierarchical Architectures	25
12	Implementation and Paper References	26
12.1	Transformer-XL	26
12.2	Compressive Transformer	26
12.3	Reformer	26
12.4	Routing Transformer	26
12.5	BigBird	27
12.6	Perceiver	27
12.7	Perceiver IO	27
12.8	Funnel Transformer	27
12.9	Synthesizer	27

Chapter 1

The Long-Context Problem

Let the input sequence be

$$X = (x_1, x_2, \dots, x_n). \quad (1.1)$$

After tokenization and embedding, the input is represented as

$$H^{(0)} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (1.2)$$

A conventional Transformer layer computes

$$Q = H^{(l)} W^Q, \quad K = H^{(l)} W^K, \quad V = H^{(l)} W^V. \quad (1.3)$$

Self-attention then computes

$$S = \frac{QK^T}{\sqrt{d_k}}, \quad (1.4)$$

followed by

$$A = \text{softmax}(S), \quad (1.5)$$

and

$$H_{\text{attn}} = AV. \quad (1.6)$$

The problem is that

$$S \in \mathbb{R}^{n \times n}. \quad (1.7)$$

Consequently, increasing sequence length produces a quadratic increase in the number of pairwise interactions.

Several Transformer variants therefore modify the architecture at a higher level.

Rather than changing only the attention formula, they change how information is organized across:

1. positions,

2. segments,
3. layers,
4. memory states,
5. local and global representations.

This part focuses on such architectures.

Chapter 2

Transformer-XL

2.1 Why Transformer-XL Was Introduced

A conventional Transformer processes a finite context.

Suppose the sequence is divided into segments:

$$X_1, X_2, \dots, X_T. \quad (2.1)$$

A conventional Transformer processes each segment independently.

Thus, when processing

$$X_t, \quad (2.2)$$

the model cannot directly access the hidden representations computed for

$$X_{t-1}. \quad (2.3)$$

Transformer-XL introduces a recurrent memory mechanism that allows hidden states from previous segments to be reused.

Paper:

Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context

2.2 Input Segmentation

Let a long sequence be divided into segments:

$$X = [X_1, X_2, \dots, X_T]. \quad (2.4)$$

Each segment contains L tokens.

For segment t ,

$$X_t = (x_{t,1}, \dots, x_{t,L}). \quad (2.5)$$

The hidden representation produced for the segment is

$$H_t = (h_{t,1}, \dots, h_{t,L}). \quad (2.6)$$

The central modification is that the next segment can reuse information from the previous segment.

2.3 Memory Construction

Suppose layer l produces

$$H_\tau^{(l)}. \quad (2.7)$$

For the next segment, a memory is constructed from previous hidden states:

$$M_\tau^{(l)} = \text{SG} \left(H_{\tau-1}^{(l)} \right), \quad (2.8)$$

where

$$\text{SG} \quad (2.9)$$

denotes stop-gradient.

The memory is therefore reused as a source of information but is not backpropagated through indefinitely.

2.4 Concatenated Memory and Current State

For the current segment, the model constructs

$$\tilde{H}_\tau^{(l)} = [M_\tau^{(l)}; H_\tau^{(l)}]. \quad (2.10)$$

Queries are generated from the current hidden states:

$$Q_\tau = H_\tau^{(l)} W^Q. \quad (2.11)$$

Keys and values are generated from the combined memory and current states:

$$K_\tau = \tilde{H}_\tau^{(l)} W^K, \quad (2.12)$$

$$V_\tau = \tilde{H}_\tau^{(l)} W^V. \quad (2.13)$$

The attention operation becomes

$$A_\tau = \text{softmax} \left(\frac{Q_\tau K_\tau^T}{\sqrt{d_k}} \right), \quad (2.14)$$

followed by

$$H_{\tau, \text{out}} = A_\tau V_\tau. \quad (2.15)$$

The important architectural change is therefore

$$\boxed{K, V \leftarrow [\text{previous memory; current segment}]}$$
 (2.16)

while

$$\boxed{Q \leftarrow \text{current segment.}}$$
 (2.17)

2.5 Relative Positional Encoding

Transformer-XL also changes the treatment of positional information.

The attention score contains content-dependent and position-dependent components.

A conceptual form is

$$A_{i,j} \propto \exp \left(\frac{q_i^T k_j}{\sqrt{d_k}} + \text{relative-position term} \right). \quad (2.18)$$

The position is therefore represented relative to the interacting query and key positions rather than treating every absolute position as an isolated index.

2.6 What Changed Relative to the Baseline

The original Transformer has

$$X_t \rightarrow H_t \rightarrow \text{attention within } X_t. \quad (2.19)$$

Transformer-XL changes this to

$$(X_t, M_{t-1}) \rightarrow \text{attention} \rightarrow H_t \rightarrow M_t. \quad (2.20)$$

Thus the architecture contains a recurrent information pathway between segments.

2.7 Applications

Transformer-XL was designed particularly for long-context language modeling.

The mechanism is useful when information from earlier portions of a sequence should remain available without recomputing all earlier hidden states.

Repository:

[Transformer-XL implementation](#)

Chapter 3

Compressive Transformer

3.1 Motivation

Transformer-XL retains previous hidden states in memory.

However, memory has finite capacity.

Suppose the memory contains

$$M_t = [m_1, m_2, \dots, m_L]. \quad (3.1)$$

If the model continually receives new segments, the memory must eventually discard older states.

The Compressive Transformer introduces a second memory that stores compressed representations of older information.

Paper:

Compressive Transformers for Long-Range Sequence Modelling

3.2 Two Memory Levels

The architecture maintains:

$$M_t \quad (3.2)$$

for recent memory, and

$$C_t \quad (3.3)$$

for compressed long-term memory.

The recent memory retains high-resolution representations.

Older states are compressed before entering the compressed memory.

3.3 Compression Function

Suppose a sequence of hidden states is

$$H = [h_1, h_2, \dots, h_n]. \quad (3.4)$$

A compression function

$$f_c \tag{3.5}$$

maps several representations to a smaller representation:

$$c_i = f_c(h_{ri}, h_{ri+1}, \dots, h_{ri+r-1}), \tag{3.6}$$

where r is a compression ratio.

A simple conceptual example is average pooling:

$$c_i = \frac{1}{r} \sum_{j=0}^{r-1} h_{ri+j}. \tag{3.7}$$

The actual architecture can use learned compression mechanisms.

3.4 Attention over Multiple Memories

Queries originate from the current segment:

$$Q = HW^Q. \tag{3.8}$$

Keys and values can be constructed from both memories:

$$K = [M_t; C_t]W^K, \tag{3.9}$$

$$V = [M_t; C_t]W^V. \tag{3.10}$$

Attention then becomes

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right), \tag{3.11}$$

with output

$$Z = AV. \tag{3.12}$$

The model therefore has access to both:

$$\text{recent high-resolution information} \tag{3.13}$$

and

$$\text{older compressed information.} \tag{3.14}$$

3.5 Information Flow

The conceptual progression is

$$H_t \rightarrow M_t \rightarrow C_t \tag{3.15}$$

for increasingly old information.

Thus:

$$\boxed{\text{current} \rightarrow \text{recent memory} \rightarrow \text{compressed long-term memory}}. \tag{3.16}$$

3.6 Applications

The architecture is intended for very long sequential dependencies.

Applications include:

- language modeling,
- long documents,
- long temporal sequences,
- sequential reasoning,
- tasks requiring information far outside the immediate context.

Chapter 4

Reformer

4.1 Motivation

The Reformer combines several modifications intended to make Transformers substantially more memory efficient.

Paper:

Reformer: The Efficient Transformer

Two particularly important mechanisms are:

1. locality-sensitive hashing attention;
2. reversible residual layers.

4.2 Locality-Sensitive Hashing Attention

Standard attention compares every query with every key.

Reformer instead hashes similar vectors into the same buckets.

Let a key vector be

$$k_i \in \mathbb{R}^d. \tag{4.1}$$

A locality-sensitive hash function maps it to a bucket:

$$b_i = h(k_i). \tag{4.2}$$

Vectors likely to be similar are designed to have a higher probability of receiving the same hash.

After hashing, tokens are sorted according to their bucket.

Attention is then computed primarily among tokens belonging to the same or neighboring buckets.

4.3 Attention after Hashing

Let

$$B_r \tag{4.3}$$

denote a bucket.

For a query q_i assigned to bucket B_r , attention is computed only over an appropriate candidate set

$$\mathcal{C}(i) \subseteq \{1, \dots, n\}. \quad (4.4)$$

The attention becomes

$$A_{ij} = 0 \quad \text{if} \quad j \notin \mathcal{C}(i). \quad (4.5)$$

For allowed positions,

$$A_{ij} = \frac{\exp(q_i^T k_j / \sqrt{d_k})}{\sum_{r \in \mathcal{C}(i)} \exp(q_i^T k_r / \sqrt{d_k})}. \quad (4.6)$$

This reduces the number of pairwise comparisons.

4.4 Reversible Residual Layers

An ordinary Transformer layer stores intermediate activations for backpropagation.

Reformer uses reversible layers so that previous activations can be reconstructed from later activations.

Split the input into two parts:

$$x = (x_1, x_2). \quad (4.7)$$

Define

$$y_1 = x_1 + F(x_2), \quad (4.8)$$

and

$$y_2 = x_2 + G(y_1). \quad (4.9)$$

The inverse transformation is

$$x_2 = y_2 - G(y_1), \quad (4.10)$$

followed by

$$x_1 = y_1 - F(x_2). \quad (4.11)$$

Thus the input activations can be reconstructed from the output activations.

This reduces the amount of activation memory required during training.

Repository:

[Trax implementation](#)

Chapter 5

Routing Transformer

5.1 Motivation

Routing Transformer attempts to create sparse attention based on content similarity rather than a fixed positional window.

Paper:

Efficient Content-Based Sparse Attention with Routing Transformers

The fundamental idea is to route tokens to clusters and perform attention primarily within those clusters.

5.2 Token Representations

Let

$$H = [h_1, \dots, h_n] \tag{5.1}$$

be the input representations.

A clustering or routing mechanism maps each token to a cluster:

$$c_i = r(h_i). \tag{5.2}$$

Thus,

$$c_i \in \{1, \dots, C\}. \tag{5.3}$$

5.3 Cluster-Based Attention

Define

$$\mathcal{C}_r = \{i : c_i = r\}. \tag{5.4}$$

A query at position i attends primarily to positions belonging to its cluster.

Thus the candidate set becomes

$$\mathcal{N}(i) = \mathcal{C}_{c_i}. \quad (5.5)$$

Attention is then

$$A_{ij} = 0 \quad j \notin \mathcal{N}(i), \quad (5.6)$$

while for routed positions,

$$A_{ij} = \text{softmax}_{j \in \mathcal{N}(i)} \left(\frac{q_i^T k_j}{\sqrt{d_k}} \right). \quad (5.7)$$

The sparsity pattern is therefore determined by the content of the representations.

5.4 Difference from Local Attention

Local attention chooses neighbors using position:

$$|i - j| \leq w. \quad (5.8)$$

Routing attention chooses neighbors according to representation similarity:

$$h_i \sim h_j. \quad (5.9)$$

Thus the two mechanisms encode different assumptions.

Chapter 6

BigBird

6.1 Motivation

BigBird combines several sparse attention patterns to retain important long-range interactions while avoiding full quadratic attention.

Paper:

Big Bird: Transformers for Longer Sequences

The attention pattern combines:

1. local attention;
2. random attention;
3. global attention.

6.2 Local Attention

Each token attends to nearby tokens.

For a window w ,

$$j \in \mathcal{N}_{\text{local}}(i) \tag{6.1}$$

when

$$|i - j| \leq w. \tag{6.2}$$

6.3 Random Attention

A token also attends to a small number of randomly selected positions.

Let

$$\mathcal{N}_{\text{random}}(i) \tag{6.3}$$

denote the random connections.

6.4 Global Attention

Special global tokens can attend broadly across the sequence.

Let

$$\mathcal{G} \tag{6.4}$$

be the global-token set.

The complete candidate set is approximately

$$\mathcal{N}(i) = \mathcal{N}_{\text{local}}(i) \cup \mathcal{N}_{\text{random}}(i) \cup \mathcal{G}. \tag{6.5}$$

Attention is computed only over this sparse set.

6.5 Why the Combination Matters

Local connections provide nearby information.

Random connections provide shortcuts between distant regions.

Global tokens provide hubs through which information can travel.

The resulting structure can be viewed as

$$\boxed{\text{local} + \text{random} + \text{global}}. \tag{6.6}$$

This produces a sparse communication graph while maintaining long-range connectivity.

6.6 Applications

BigBird was developed particularly for long sequences such as

- documents,
- long-form text,
- question answering,
- summarization,
- genomic sequences.

Repository:

[BigBird implementation](#)

Chapter 7

Perceiver

7.1 Motivation

A conventional Transformer applies self-attention directly to the input sequence.

If

$$X \in \mathbb{R}^{n \times d}, \tag{7.1}$$

self-attention creates interactions involving the sequence length n .

The Perceiver architecture introduces a fixed-size latent array that receives information from the input.

Paper:

Perceiver: General Perception with Iterative Attention

7.2 Input and Latent Array

Let the input be

$$X \in \mathbb{R}^{n \times d_x}. \tag{7.2}$$

Introduce a latent representation

$$Z \in \mathbb{R}^{m \times d_z}, \tag{7.3}$$

where

$$m \ll n. \tag{7.4}$$

The crucial idea is that the expensive cross-input interaction operates through m latent vectors rather than requiring self-attention across all n input elements.

7.3 Cross-Attention

Queries are generated from the latent array:

$$Q = ZW^Q. \quad (7.5)$$

Keys and values are generated from the input:

$$K = XW^K, \quad (7.6)$$

$$V = XW^V. \quad (7.7)$$

The latent array attends to the input:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right). \quad (7.8)$$

Since

$$Q \in \mathbb{R}^{m \times d_k} \quad (7.9)$$

and

$$K \in \mathbb{R}^{n \times d_k}, \quad (7.10)$$

the attention matrix has size

$$m \times n. \quad (7.11)$$

The latent representation becomes

$$Z' = AV. \quad (7.12)$$

7.4 Latent Self-Attention

After receiving information from the input, the latent array can perform self-attention:

$$Q_z = Z'W_z^Q, \quad (7.13)$$

$$K_z = Z'W_z^K, \quad (7.14)$$

$$V_z = Z'W_z^V. \quad (7.15)$$

Then

$$Z'' = \text{softmax} \left(\frac{Q_z K_z^T}{\sqrt{d_k}} \right) V_z. \quad (7.16)$$

Because the latent size m is fixed and small, the repeated latent self-attention costs

$$O(m^2) \quad (7.17)$$

rather than

$$O(n^2). \tag{7.18}$$

7.5 Iterative Processing

The architecture can repeatedly apply latent self-attention:

$$Z_0 \rightarrow Z_1 \rightarrow Z_2 \rightarrow \cdots \rightarrow Z_L. \tag{7.19}$$

The input is therefore first compressed into a learned latent representation and then processed in latent space.

7.6 Applications

Perceiver was designed for multimodal inputs, including:

- images,
- audio,
- video,
- text,
- multimodal data.

Repository:

[Perceiver implementation](#)

Chapter 8

Perceiver IO

8.1 Motivation

The original Perceiver provides a latent representation of the input.

However, many tasks require structured outputs whose size and meaning differ from the latent representation.

Perceiver IO introduces a flexible query mechanism for producing outputs.

Paper:

Perceiver IO: A General Architecture for Structured Inputs and Outputs

8.2 Latent Processing

The input is first mapped into latent space:

$$X \rightarrow Z. \tag{8.1}$$

The latent array is processed using self-attention:

$$Z \rightarrow Z^{(1)} \rightarrow \dots \rightarrow Z^{(L)}. \tag{8.2}$$

8.3 Output Queries

For an output position r , define an output query

$$q_r. \tag{8.3}$$

This query can represent the desired output location or task.

The query attends to the latent representation:

$$Q_r = q_r W^Q, \tag{8.4}$$

$$K = Z W^K, \tag{8.5}$$

$$V = ZW^V. \quad (8.6)$$

The output attention is

$$A_r = \text{softmax} \left(\frac{Q_r K^T}{\sqrt{d_k}} \right). \quad (8.7)$$

The output representation is

$$y_r = A_r V. \quad (8.8)$$

Thus different output queries can extract different information from the same latent representation. The complete mapping becomes

$$\boxed{X \rightarrow Z \rightarrow \text{output queried from } Z}. \quad (8.9)$$

Chapter 9

Funnel Transformer

9.1 Motivation

A standard Transformer maintains the same sequence length through the network.

If

$$H^{(l)} \in \mathbb{R}^{n \times d}, \quad (9.1)$$

then the next layer also receives approximately n token representations.

Funnel Transformer progressively reduces the number of hidden states in deeper layers.

Paper:

Funnel-Transformer: Filtering out Sequential Redundancy for Efficient Language Processing

9.2 Pooling

Suppose a layer produces

$$H = [h_1, h_2, \dots, h_n]. \quad (9.2)$$

A pooling operation can combine neighboring representations.

For example,

$$h'_i = \frac{1}{2}(h_{2i-1} + h_{2i}). \quad (9.3)$$

The new sequence is

$$H' = [h'_1, \dots, h'_{n/2}]. \quad (9.4)$$

Thus

$$n \rightarrow \frac{n}{2}. \quad (9.5)$$

9.3 Attention after Downsampling

The next Transformer block operates on

$$H' \in \mathbb{R}^{(n/2) \times d}. \quad (9.6)$$

Consequently, the attention matrix becomes

$$A' \in \mathbb{R}^{(n/2) \times (n/2)}. \quad (9.7)$$

Its size is one quarter of the original $n \times n$ matrix.

If repeated downsampling is applied,

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \cdots. \quad (9.8)$$

This progressively reduces computational cost.

9.4 Decoder Reconstruction

For token-level tasks, the reduced representation can be projected back toward the original resolution.

Conceptually,

$$H_{\text{short}} \rightarrow H_{\text{expanded}}. \quad (9.9)$$

The architecture therefore resembles a funnel:

$$\boxed{\text{long sequence} \rightarrow \text{compressed representation} \rightarrow \text{task output}}. \quad (9.10)$$

9.5 Applications

Funnel Transformer is particularly useful when many token-level representations contain redundancy but the final task does not require every intermediate layer to preserve the original sequence length.

Chapter 10

Synthesizer

10.1 Motivation

Standard self-attention computes pairwise token interactions through

$$QK^T. \tag{10.1}$$

Synthesizer investigates whether attention weights must actually be generated from query-key similarity.

Paper:

Synthesizer: Rethinking Self-Attention in Transformer Models

The central idea is to generate the mixing weights using alternative mechanisms.

10.2 Random Synthesizer

A random attention matrix can be generated independently of the input content.

Conceptually,

$$A = \text{softmax}(R), \tag{10.2}$$

where

$$R \in \mathbb{R}^{n \times n} \tag{10.3}$$

is a fixed or randomly generated matrix.

The output is

$$Z = AV. \tag{10.4}$$

10.3 Dense Synthesizer

Instead of computing

$$QK^T, \tag{10.5}$$

the attention weights can be generated directly from the input.
For example,

$$R = f(H), \tag{10.6}$$

where

$$f : \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{n \times n}. \tag{10.7}$$

Then

$$A = \text{softmax}(R), \tag{10.8}$$

and

$$Z = AV. \tag{10.9}$$

The key difference is that the mixing matrix is not explicitly constructed as a query-key similarity matrix.

Chapter 11

Efficient Transformer Design as a Spectrum

The architectures discussed in this part can be organized according to what they change.

11.1 Memory-Based Architectures

Transformer-XL introduces explicit recurrence:

$$\text{segment}_t + \text{memory}_{t-1} \rightarrow \text{segment}_t. \quad (11.1)$$

Compressive Transformer extends this with compressed memory:

$$\text{recent memory} + \text{compressed memory}. \quad (11.2)$$

11.2 Sparse Attention

Reformer, Routing Transformer, and BigBird reduce the number of attention connections.

Instead of

$$n^2 \quad (11.3)$$

interactions, only a subset

$$|\mathcal{E}| \ll n^2 \quad (11.4)$$

is computed.

11.3 Latent Bottleneck Architectures

Perceiver replaces direct input-to-input attention with

$$X \rightarrow Z \rightarrow Z, \quad (11.5)$$

where

$$|Z| \ll |X|. \quad (11.6)$$

This introduces a learned information bottleneck.

11.4 Hierarchical Architectures

Funnel Transformer progressively changes the representation resolution:

$$n_0 > n_1 > n_2 > \cdots. \quad (11.7)$$

The Transformer therefore does not maintain the same token resolution at every layer.

Chapter 12

Implementation and Paper References

12.1 Transformer-XL

Paper:

[Transformer-XL](#)

Code:

<https://github.com/kimiyoung/transformer-xl>

12.2 Compressive Transformer

Paper:

[Compressive Transformers for Long-Range Sequence Modelling](#)

12.3 Reformer

Paper:

[Reformer: The Efficient Transformer](#)

Code:

<https://github.com/google/trax>

12.4 Routing Transformer

Paper:

[Efficient Content-Based Sparse Attention with Routing Transformers](#)

12.5 BigBird

Paper:

[Big Bird: Transformers for Longer Sequences](#)

Code:

<https://github.com/google-research/bigbird>

12.6 Perceiver

Paper:

[Perceiver: General Perception with Iterative Attention](#)

Code:

[Perceiver implementation](#)

12.7 Perceiver IO

Paper:

[Perceiver IO](#)

12.8 Funnel Transformer

Paper:

[Funnel-Transformer](#)

12.9 Synthesizer

Paper:

[Synthesizer: Rethinking Self-Attention in Transformer Models](#)